

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА ОБРАБОТКА НА ОБЛАЦИ ОТ
ТОЧКИ И КАРТИ НА ДЪЛБОЧИНАТА С
УСКОРЕНО ТЪРСЕНЕ ВЪРХУ
ПРОСТРАНСТВЕН ИНДЕКС**

КУРСОВ ПРОЕКТ

по дисциплина „Компютърно зрение“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Пространствен индекс	5
2.2. Стратегия за векторизация	5
2.3. Организация на конвейера и роля на външните библиотеки	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	8
3.1. Разположение на данните в паметта	8
3.2. Интерфейс на индекса	9
3.3. Договор на стъпката	9
3.4. Поток на данните	10
3.5. Извън обхвата	10
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	12
4.1. Облак от точки и индекс	12
4.2. Обхождане на заявката	13
4.3. Двата пътя на обхождане на лист	14
4.4. Сингулярно разлагане и оценяване на преобразуванието	16
4.5. Итерация на съвместяването	19
4.6. Прореждане с вокселна решетка	19
4.7. Вход и изход	20
4.8. Растерни стъпки	20
4.9. Проверка и измерване	21
4.10. Едно наблюдение за изграждането	21
V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА	22
5.1. Еталон за коректност	22
5.2. Данни	23
5.3. Конфигурация на машината	23

5.4. Протокол на изпълнение	24
5.5. Измервани величини и форма на таблиците	25
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	26
6.1. Съвпадение с еталона	26
6.2. Построяване на индекса	27
6.3. Цена на заявката за съседи	27
6.4. Изолирано обхождане на лист	29
6.5. Заявка по радиус и размер на листа	29
6.6. Прореждане с вокселна решетка	31
6.7. Съвместяване	31
6.8. Растерен конвейер	32
6.9. Анализ	33
ЗАКЛЮЧЕНИЕ	35
ЛИТЕРАТУРА	37
ПРИЛОЖЕНИЯ	39

УВОД

Сензорите за дълбочина и стереоскопичните камери извеждат данни, които не приличат на обикновено растерно изображение. Резултатът е *облак от точки*: неподредено множество от координати в тримерното пространство, чиито съседства не се четат от индекса на пиксела, а трябва да бъдат намерени. Почти всяка стъпка от обработката, от премахването на шума през оценяването на нормалите и извличането на признаци до съвместяването на два припокриващи се записа, се свежда до един и същ въпрос: кои точки лежат близо до дадена точка. Затова цената на целия конвейер се определя не толкова от аритметиката на отделния алгоритъм, колкото от това колко бързо се отговаря на този въпрос.

Целта на курсовия проект е да се проектира и реализира конвейер за обработка на облаци от точки и растерни изображения, в който търсенето на съседи е изнесено в отделен, ускорен компонент върху пространствен индекс, и всяка стъпка от конвейера е измерима самостоятелно. Реализацията е на C++20 и се изгражда без нито една външна зависимост. Ускорението идва от подредба на данните по компонента и от пакетно устроен вътрешен цикъл, който компилаторът превежда във векторни инструкции; че това наистина се случва, се проверява от отчета на компилатора, а не се предполага.

За постигането на целта са формулирани следните **задачи**:

1. преглед на апаратната и информационната структура на система за компютърно зрение и на класическите методи за предварителна обработка, филтриране, откриване на контури, извличане на признаци и сегментация;
2. анализ на възможните решения за пространствен индекс, за стратегия на векторизация и за организация на конвейера, и обосновка на направения избор;
3. проектиране на конвейер със самостоятелно измерими стъпки и с ясна граница между преносимата логика и векторизираната част;
4. програмна реализация на стъпките: четене и запис на облаци и изображения, прореждане с вокселна решетка, пространствен индекс със заявки за най-близки съседи и по радиус в два взаимозаменяеми варианта на вътрешния цикъл, съвместяване на облаци по схемата на итеративната най-близка точка, и растерна верига от изглаждане, градиент, потискане на немаксимуми и сегментация по свързани компоненти;
5. изграждане на методика за измерване, която сравнява всяка стъпка по точност и по

време спрямо независим еталон;

б. провеждане на измерванията и анализ на получените резултати.

Обхватът на проекта е геометричната и фотометричната обработка: всичко от входните данни до съвместените облаци и сегментираните области. Извън него остават още оценяването на дълбочина по стереодвойка и извличането на описания на околност: и двете са потребители на заявката за съседи, чиято цена работата измерва пряко, и не участват в измерваната верига. Извън обхвата съзнателно остават разпознаването по обучени модели и невронните мрежи. Причината не е липса на интерес, а несъвместима методика на оценяване: качеството на обучен разпознавател зависи преди всичко от набора от данни и от процедурата на обучение, а предмет на измерване тук е цената на геометричната обработка. Двете не се измерват с един и същ експеримент, а обем от един курсов проект не стига и за двете. Тези теми са разгледани в литературния преглед и мястото им в конвейера е посочено, но не са реализирани.

Структура на изложението. Раздел I разглежда предметната област и методите, върху които стъпва работата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на конвейера, а Раздел IV неговата програмна реализация. Раздел V описва методиката на измерване, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за продължение.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Системата за компютърно зрение се описва обичайно като верига от източник на данни, слой за предварителна обработка, слой за извличане на признаци и слой за вземане на решение. Апаратната част определя вида и дефектите на входа: пасивната стереодвойка дава плътна карта на дълбочината само там, където сцената има текстура, а активните сензори със структурирана светлина или с време на прелитане дават плътност, независима от текстурата, но внасят шум по краищата на обектите и загуба на данни при отразяващи повърхности. Информационната структура надолу по веригата е следствие от този избор: подредено изображение на дълбочината запазва съседството по индекс, докато облакът от точки го губи и налага пространствен индекс.

Предварителната обработка и филтрирането целят потискане на шума без разрушаване на геометрията. Линейното изглаждане размива именно ръбовете, които следващите стъпки търсят, затова се предпочитат методи, съобразени с ръба: двустранното филтриране претегля съседите едновременно по разстояние и по разлика в стойността [1], а при облаци от точки същата идея се прилага върху разстоянието до допирателната равнина. Откриването на контури върху изображението на интензитета следва класическата постановка на Кани с трите ѝ критерия за добро откриване, добра локализация и единичен отговор [2]; върху карта на дълбочината същият апарат откроява геометричните прекъсвания, но изисква отделно третиране на липсващите стойности.

Извличането на признаци и сегментацията са мястото, където подредените и неподредените данни се разминават. За облаци от точки утвърдено средство са хистограмите на бързите точкови признаци, които описват околността на точка чрез разпределение на ъглите между нормалите на съседите ѝ и са проектирани така, че цената им да расте линейно с броя на точките [3]. Сегментацията на еднородни области се води или от геометрията, чрез съгласуваност на нормалите и кривината, или от подгонване на параметричен модел към подмножество от точки. Вторият подход почти винаги стъпва върху устойчиво оценяване със случайни извадки [4], което остава работещо при голям дял груби грешки. Текстурната сегментация на изображението ползва отделно семейство описания, класическото сред които са статистиките от матрицата на съвместната поява на нива на сивото [5].

Анализът на тримерната сцена и стереодвойката са двете страни на един и същ въпрос за възстановяване на дълбочината. Полуглобалното съгласуване свежда двумерната задача за съответствие до сумиране на едномерни динамични програми по няколко направления и е приетият компромис между качеството на глобалните и цената на локалните методи [6]. Когато сцената се сменя от няколко положения, получените облаци трябва да бъдат приведени в обща координатна система. Каноничното решение е итеративната най-близка точка [7], чиито варианти се различават по избора на съответствия, по метриката и по стратегията на отхвърляне; систематичното им сравнение показва, че тези решения влияят на скоростта на сходимост повече от самата формулировка [8]. Всяка итерация изисква търсене на най-близък съсед, което връща изложението към структурата на данните: k-d дървото [9] остава основното средство, а приблизителните варианти купуват скорост срещу контролирана загуба на точност [10].

Разпознаването по модел и подходите с невронни мрежи заемат последното ниво на веригата и са извън реализацията по причината, изложена в увода. Тук те се отбелязват като потребител на изхода от настоящия конвейер: сегментираните области и описанията на околностите са точно входът, който един разпознавател очаква, независимо дали е геометричен или обучен. Утвърдените реализации с отворен код в областта са библиотеката за облаци от точки [11] и библиотеката за компютърно зрение [12]. Настоящата работа не ги използва и не се сравнява с тях, по причина, изложена в Раздел III: те служат за ориентир кои алгоритми са установената практика, не за еталон при измерването.

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Пространствен индекс

Три решения са приложими за търсене на съседи в облак от точки. Вокселната решетка разделя пространството на кубчета с постоянен размер и намира съседите чрез обхождане на съседните кубчета. Тя е тривиална за построяване и изключително бърза при равномерна плътност, но при неравномерна плътност или силно издължена сцена или изразходва памет за празни кубчета, или събира прекалено много точки в едно. K-d дървото [9] се приспособява към плътността, защото разделя по медиана, и дава предсказуемо поведение при заявка за k най-близки съседи, но обхождането му е с нередовен достъп до паметта. Октодървото стои между двете и се предпочита, когато е нужна работа на няколко разделителни способности едновременно.

Изборът е k-d дърво като пространствен индекс, а вокселната решетка се използва в другата си роля, като стъпка за прореждане преди индексирването. Първоначалният замисъл беше решетката да бъде и второ, независимо изпълнение на същия интерфейс, за да се измери едното срещу другото. От това е отстъпено: при неравномерна плътност, каквато има в изпитваната сцена, решетката като индекс или изразходва памет за празни кубчета, или събира прекалено много точки в едно, и измерването щеше да покаже това, което устройството ѝ и без него казва. Приблизителните схеми [10] са разгледани и оставени извън обхвата, защото внасят втори източник на грешка, който би се смесил с грешката на измервания алгоритъм.

2.2. Стратегия за векторизация

Ускоряването на заявките има три възможни пътя. Вътрешните функции на конкретния набор инструкции дават пълен контрол, но обвързват изходния текст с една архитектура и налагат отделна реализация за всяка. Библиотека за преносима векторизация запазва единен изходен текст и оставя избора на набор инструкции на времето за компилация, но е външна зависимост. Автоматичната векторизация на компилатора е най-евтина откъм усилие, но резултатът зависи от компилатора и от неговата версия и не е част от договора на езика.

Избран е третият път, с една добавка, която премахва главния му недостатък. Вътрешният цикъл е написан нарочно в пакетна форма: разстоянията за блок от точки се смятат в отделен цикъл без разклонения, без обръщение към споделено състояние и с константен брой итерации, а проверките са изнесени в следващ цикъл. Тази форма компилаторът разпознава. Че я разпознава, не се предполага, а се проверява с `-fort-info-vec`, и отговорът е записан в Раздел IV.

Решаващият довод срещу библиотеката за преносима векторизация е ограничението за нулеви задължителни зависимости, изложено по-долу. Условието изборът да работи е разположението на данните: заявките се векторизират смислено само ако координатите са подредени по компонента, а не по точка. Това превръща оформлението на паметта в проектно решение, а не в подробност от реализацията, и е разгледано в Раздел III.

2.3. Организация на конвейера и роля на външните библиотеки

Конвейерът може да бъде написан като монолитна функция, като верига от обекти с общ интерфейс, или като съвкупност от независими преобразувания, свързвани при конфигурирането. Първото е най-кратко, но прави отделната стъпка неизмерима. Избрано е второто: всяка стъпка е обект с еднакъв договор, който приема облак и връща облак заедно с показатели за изпълнението си. Така времето на стъпката е наблюдаемо без промяна в кода, а редът на стъпките се задава отвън.

Разгледани бяха две положения за външните библиотеки. Първото е те да поемат входа и изхода на данните и да служат за еталон при сравнението [11, 12]. Второто, избраното, е проектът да няма нито една задължителна външна зависимост.

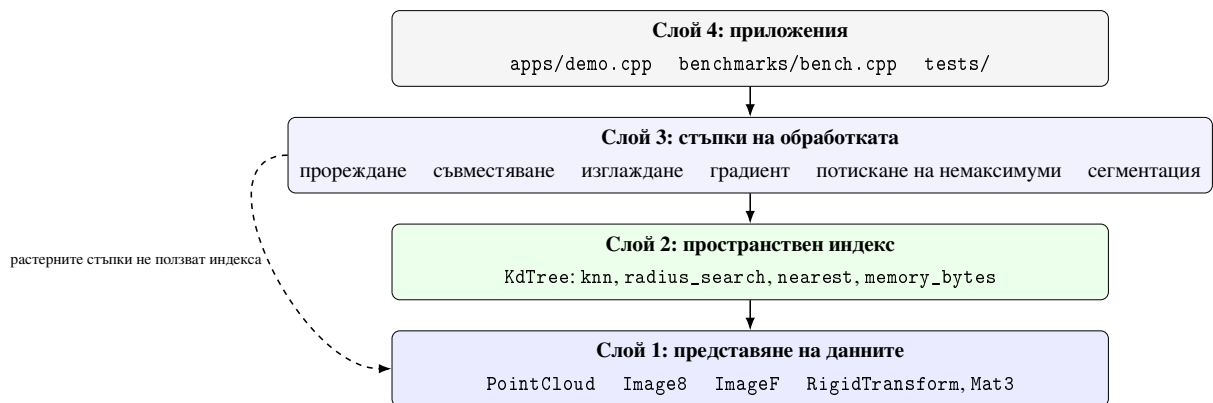
Доводът за второто е практически. Хранилището е публично, а изграждане, което изисква мениджър на пакети, на практика не се изпълнява от никого при първия опит. Цената на избора е обозрима и е платена: четците на PLY, PCD, PGM и PPM, рамката за проверка, измервателната част и разлагането на матрица 3x3 са написани в проекта, общо под хиляда реда. Загубеното е сравнението с чужда реализация. То е заместено с еталон, който не изисква зависимост и е по-силен по своя въпрос: изчерпателното търсене, което е точно по определение, и известното кораво преобразуване, срещу което се проверява съвместяването.

Алгоритмите, които са предмет на изследването, и при двете положения се

реализират в проекта. Обратното решение, тоест извикване на готовите реализации, би направило измерването безсмислено, защото щеше да сравнява библиотека със самата себе си.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е разделена на четири слоя. Най-долният съдържа представянето на данните: облак от точки и растерно изображение. Над него стои пространственият индекс с единен интерфейс за заявка по k най-близки съседи и за заявка по радиус. Третият слой съдържа стъпките на обработката, всяка от които зависи само от долните два. Най-горният слой са приложенията: демонстрацията, измерванията и проверките. Разделянето е показано на Фиг. 1.



Фигура 1. Слоевете на системата. Зависимостите сочат само надолу.

3.1. Разположение на данните в паметта

Разположението на данните е проектно решение, не подробност. Облакът се съхранява като три отделни непрекъснати масива, по един за всяка координата, а не като един масив от тройки. Класът `PointCloud` пази `std::vector<float>` за x , за y и за z и дава пряк достъп до трите указателя, защото векторизираният път ги ползва направо.

Причината е достъпът при заявка. Вътрешният цикъл на обхождането на лист чете една координата на много точки последователно. При разположение по компонента това е последователен достъп с крачка 4 байта, тоест едно четене на кеш-линия обслужва шестнадесет точки. При масив от тройки крачката е 12 байта и същият цикъл докосва три пъти повече кеш-линии, а векторизацията изисква разместване на регистрите. Цената на избора е по-неудобен достъп до отделна точка: методът `point(i)` събира тройката от три различни масива. Затова той е на границата на интерфейса, а не във вътрешните цикли.

3.2. Интерфейс на индекса

Интерфейсът на индекса е умишлено тесен: построяване от облак, заявка за k най-близки съсед, заявка по радиус, заявка за единствен най-близък съсед и отчет за паметта. Всичко останало в системата е потребител на този интерфейс, а не негова част.

Двата пътя за обхождане на лист, *скаларен* и *пакетен*, не са две различни реализации на индекса, а параметър на една и съща заявка. Решението е методическо. Ако бяха два класа или два варианта на компилация, сравнението между тях щеше да сравнява и всичко останало, което ги отличава: различни дървета, различно подреждане на паметта, евентуално различни версии на кода. Като параметър на метода те работят върху едно и също дърво, върху една и съща подредба, в едно и също изпълнение на програмата, и единствената разлика между двете измервания е тялото на цикъла. Това е условието сравнението да е измерване, а не твърдение.

Заявката за единствен най-близък съсед е отделен метод, а не заявка с $k = 1$. Причината е потребителят i : съвместяването вика точно нея веднъж на точка на всяка итерация, тоест милиони пъти, и купчината с кандидати, която общият път поддържа, там е чиста излишна работа. Двата метода дават един и същ резултат и това се проверява.

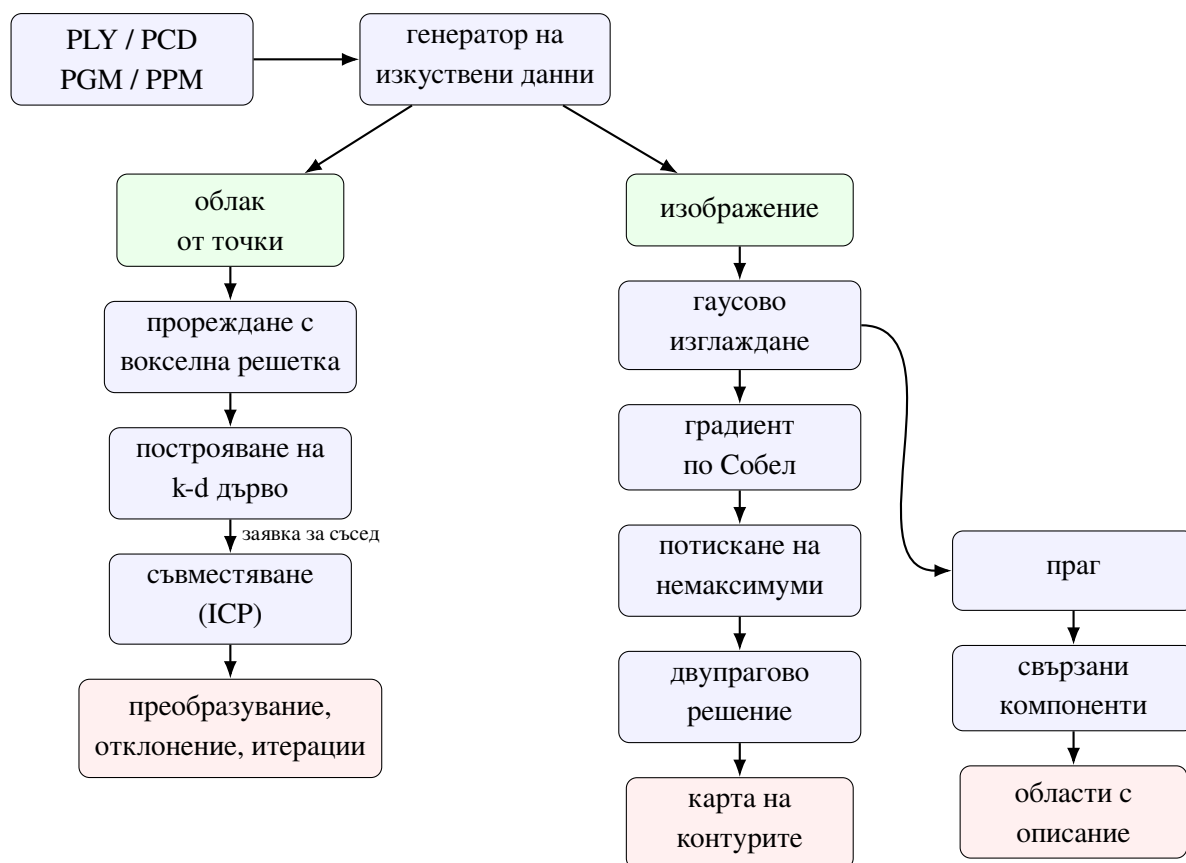
3.3. Договор на стъпката

Всяка стъпка приема вход и конфигурация и връща нов резултат. Стъпките не променят входа си на място. Решението е за сметка на паметта, но има три последици, които го оправдават: повтарянето на една стъпка с различни параметри не изисква повторно изпълнение на предходните, отпада скрита зависимост по ред на изпълнение, която би направила измерването на отделна стъпка невярно, и стъпката става безопасна за паралелно изпълнение върху различни входове, без това да е било предвиждано отделно.

Конфигурацията е структура с полета по подразбиране, а не файл. Разгледан беше и конфигурационен файл, който конвейерът да чете при пускане. Отхвърлен е, защото би въвел или зависимост от библиотека за разбор на формата, или собствен разбор, който е код без връзка с предмета на работата. Структурите `KdTreeOptions`, `IcpOptions`, `SceneOptions` и `ImageSceneOptions` дават същото, което файлът би дал, а стойностите по подразбиране са на едно място в заглавния файл, където се четат заедно с описанието на полето.

3.4. Поток на данните

Потокът е показан на Фиг. 2. Геометричният и растерният клон са независими: те споделят слоевете отдолу, но никоя стъпка от единия не е вход на стъпка от другия. Разделянето е следствие от обхвата: работата изследва цената на заявката за съсед, а тя се среща само в геометричния клон.



Фигура 2. Поток на данните. Геометричният клон е отляво, растерният отдясно.

3.5. Извън обхвата

Три решения от първоначалния замисъл са отпаднали и е редно да бъде записано защо, а не да останат като липса.

Втора реализация на индекса, вокселна решетка като заменяем пространствен индекс, е изоставена. Вокселната решетка е реализирана, но като *стъпка за прореждане*, което е ролята, в която тя носи полза в този конвейер. Като индекс тя би трябвало да отговаря на същите заявки като k-d дървото, а при неравномерна плътност, каквато има в изпитваната сцена, това означава или памет за празни кубчета, или прекалено много точки в едно. Сравнението между двете щеше да измерва това, което Раздел II вече установява

от устройството им.

Сравнение с еталонна библиотека с отворен код е изоставено. Причината е ограничението за изграждане: проектът се компилира с компилатор за C++20 и CMake и с нищо друго, а сравнението би добавило задължителна зависимост заради една таблица. Еталонът за коректност на индекса е изчерпателното търсене, което е точно по определение и е реализирано в проекта.

Оценяването на дълбочина по стереодвойка и извличането на описания на околност са изоставени по обем. Те не участват в измерваната верига: и двете са потребители на заявката за съсед, а нейната цена вече се измерва пряко.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Изходният текст е на C++20 и се изгражда с CMake, версия 3.20 или по-нова. Публичните типове са в заглавни файлове под `include/pointforge/`, а реализацията е в `src/`, така че смяна на реализацията на индекса не води до преизграждане на целия проект. Разпределението на кода по файлове е дадено в Табл. 1.

Таблица 1. Модули на реализацията

Файл	Редове	Съдържание
<code>point_cloud.hpp</code>	153	облак от точки, разположен по компонента; ограждащ паралелепипед
<code>transform.{hpp,cpp}</code>	305	матрица 3x3, сингулярно разлагане, кораво преобразуване
<code>kdtree.{hpp,cpp}</code>	605	построяване на дървото, трите вида заявка, двата пътя на лист
<code>icp.{hpp,cpp}</code>	257	съответствия, отхвърляне, оценяване по Кабш, итерация
<code>voxel_grid.{hpp,cpp}</code>	149	прореждане с вокселна решетка
<code>cloud_io.{hpp,cpp}</code>	307	четене и запис на PLY и PCD
<code>image.{hpp,cpp}</code>	179	двата растерни типа и преобразуванията между тях
<code>image_io.{hpp,cpp}</code>	163	четене и запис на PGM и PPM
<code>image_ops.{hpp,cpp}</code>	238	изглаждане, градиент, потискане на немаксимуми, прагове
<code>labeling.{hpp,cpp}</code>	179	свързани компоненти и описанията им
<code>synthetic.{hpp,cpp}</code>	272	генератор на изкуствени облаци и изображения
<code>timing.hpp</code>	80	измерване на време без външна библиотека
<code>text.hpp</code>	37	подравняване на изхода при кирилица

4.1. Облак от точки и индекс

Класът `PointCloud` пази три вектора от `float`, по един на координата, и дава както достъп до отделна точка чрез `point(i)`, така и пряк достъп до указателите `xs()`, `ys()` и `zs()`. Вторият е част от договора, а не пробив в капсулацията: без него пакетният път не

може да бъде написан така, че компилаторът да го векторизира.

Построяването на дървото в `KdTree::build` върви върху масив от индекси, а не върху самите точки. Разделянето на всяко ниво се прави с `std::nth_element` по медианата на най-дългата ос на текущото подмножество. Изборът на най-дълга ос, вместо редуване на осите по нива, има значение при сцена, в която една ос е много по-къса от другите: редуването тогава дава дълги тесни клетки, при които разстоянието до разделящата равнина рядко надхвърля текущото най-добро и отсичането не работи.

Координатите се разбъркват в реда на обхождане чак накрая, наведнъж. Алтернативата, разместване при всяко разделяне, би струвала по три записа на точка на всяко ниво на дървото. След разбъркването точките на един лист са съседни в паметта, което е предпоставката пакетният път изобщо да има смисъл.

Едно място в построяването не е очевидно и заслужава да бъде посочено. Ако всички точки в подмножеството имат еднаква стойност по избраната ос, разделянето по медиана не напредва: и лявата, и дясната половина съдържат същите точки и рекурсията не свършва. Проверката е дали медианата съвпада едновременно с първия и с последния елемент след `nth_element`; при съвпадение се създава лист, дори по-голям от зададения размер. Случаят не е измислен: облак, легнал точно върху равнина, го предизвиква по оста, по която е плосък.

4.2. Обхождане на заявката

Обхождането е итеративно, със собствен стек, а не рекурсивно. Стекът пази двойка от номер на възел и квадрат на разстоянието до равнината, която го е отделила от вече обходения клон. Така отсичането се прави при изваждането от стека, без повторно пресмятане: ако това разстояние вече е по-голямо от най-лошия приет кандидат, поддървото се пропуска изцяло.

Кандидатите се държат в купчина с най-лошия елемент отгоре, ограничена до k елемента, реализирана със `std::push_heap` и `std::pop_heap` върху вектора на резултата. Векторът се подава от повикващия и се използва между заявките. Причината е потребителят: съвместяването прави по една заявка на точка на всяка итерация, тоест милиони заявки, и заделянето на памет за всяка от тях доминира над самото търсене.

4.3. Двата пътя на обхождане на лист

Това е мястото, около което е построена цялата работа. Скаларният път обхожда точките на листа една по една и след всяка проверява дали кандидатът влиза в купчината (Листинг 1).

```
153 void scan_leaf_scalar(const LeafScanContext& c, std::uint32_t begin, std:::
    ↪ uint32_t end,
154                     BoundedHeap& heap) {
155     for (std::uint32_t i = begin; i < end; ++i) {
156         const float dx = c.px[i] - c.qx;
157         const float dy = c.py[i] - c.qy;
158         const float dz = c.pz[i] - c.qz;
159         const float d2 = dx * dx + dy * dy + dz * dz;
160         if (d2 < heap.worst() || !heap.full()) heap.offer(c.original[i], d2);
161     }
162 }
```

Листинг 1: Скаларно обхождане на лист, src/kdtree.cpp

Пакетният път разделя същата работа на два цикъла (Листинг 2). Първият смята квадратите на разстоянията за цял блок от `kBlockSize` точки в масив на стека. В него няма разклонения, няма обръщение към купчината и няма зависимост между итерациите, а трите четения са последователни. Точно това е формата, която компилаторът разпознава. Вторият цикъл прави проверките и обновяванията.

Между двата стои проверка, която не е оптимизация на дребно: ако купчината е пълна и дори най-близката точка от блока е по-далече от най-лошата приета, целият блок се пропуска. При запълнена купчина това спестява повечето обръщения към нея, а именно те са скъпата част. Разделянето на цикъла с фиксиран брой итерации при пълен блок и отделен цикъл за остатък също е нарочно: с променлив горен предел компилаторът добавя проверка за остатък, а с константен разгъва цикъла напълно.

```
174 void scan_leaf_batched(const LeafScanContext& c, std::uint32_t begin, std:::
    ↪ uint32_t end,
175                     BoundedHeap& heap) {
176     float distances[KdTree::kBlockSize];
177 }
```

```

178     for (std::uint32_t base = begin; base < end; base += KdTree::kBlockSize)
↪ {
179         const std::uint32_t n = std::min<std::uint32_t>(KdTree::kBlockSize,
↪ end - base);

180         const float* bx = c.px + base;
181         const float* by = c.py + base;
182         const float* bz = c.pz + base;
183
184         if (n == KdTree::kBlockSize) {
185             for (std::uint32_t j = 0; j < KdTree::kBlockSize; ++j) {
186                 const float dx = bx[j] - c.qx;
187                 const float dy = by[j] - c.qy;
188                 const float dz = bz[j] - c.qz;
189                 distances[j] = dx * dx + dy * dy + dz * dz;
190             }
191         } else {
192             for (std::uint32_t j = 0; j < n; ++j) {
193                 const float dx = bx[j] - c.qx;
194                 const float dy = by[j] - c.qy;
195                 const float dz = bz[j] - c.qz;
196                 distances[j] = dx * dx + dy * dy + dz * dz;
197             }
198         }
199
200         if (heap.full()) {
201             float block_min = distances[0];
202             for (std::uint32_t j = 1; j < n; ++j) block_min = std::min(
↪ block_min, distances[j]);
203             if (block_min >= heap.worst()) continue;
204         }
205
206         for (std::uint32_t j = 0; j < n; ++j) {
207             if (distances[j] < heap.worst() || !heap.full()) heap.offer(c.
↪ original[base + j], distances[j]);
208         }
209     }
210 }

```

Листинг 2: Пакетно обхождане на лист, src/kdtree.cpp

Че цикълът наистина се векторизира, е проверено, а не предположено. Компиляторът се пита пряко:

```
g++ -std=c++20 -O3 -Iinclude -c src/kdtree.cpp \
-o /dev/null -fopt-info-vec
```

При подразбирация се набор инструкции отговорът за редовете на пакетния път е `loop vectorized using 16 byte vectors`, тоест четири числа с плаваща запетая наведнъж. С `-march=native` на изпитваната машина същите редове дават `32 byte vectors`, тоест осем. Редовете на скаларния път не се появяват в отчета при никоя от двете настройки.

4.4. Сингулярно разлагане и оценяване на преобразуванието

Съвместяването изисква разлагане на матрица 3×3 . Вместо библиотека за линейна алгебра е написан едностранен метод на Якоби в `svd3`: върти двойки стълбове на копие на матрицата, докато станат взаимно ортогонални, след което колонните норми са сингулярните числа. Методът е предпочетен пред разлагане на $A^T A$, защото последното повдига числото на обусловеност на квадрат, а тук матрицата е кръстосана ковариация между два облака и при почти плосък облак вече е зле обусловена.

Едно място в разлагането се оказа източник на грешка и е редно да бъде описано, а не изгладено. Когато матрицата е с ранг под три, съответният стълб на U не е определен от входа: нормата му излиза нула и няма какво да бъде нормирано. Първата реализация оставяше на негово място съответния единичен вектор. Това изглежда безобидно, но прави U неортогонална, а произведението VU^T тогава понякога излиза *отражение*, не ротация. Проверката `icp.kabsch_never_returns_a_reflection`, която подава точки върху равнина, го улови. Поправката е стълбът да бъде *допълнен* до ортонормиран базис чрез векторно произведение на другите два.

Самото оценяване на преобразуванието (Листинг 3) следва Кабш: центрове на тежестта, кръстосана ковариация, разлагане, $R = VU^T$, и поправка на знака при отрицателна детерминанта чрез смяна на знака на стълба, съответстващ на най-малкото сингулярно число. Сумирането на центровете е в `double`, макар точките да са `float`: при облак, отместен далече от началото на координатната система, натрупването във `float` губи точно значещите цифри, които носят формата на облака.

```

43 RigidTransform estimate_rigid_transform(const PointCloud& source, const
↳ PointCloud& target,
44                                     const std::vector<std::uint32_t>&
↳ source_indices,
45                                     const std::vector<std::uint32_t>&
↳ target_indices) {
46     RigidTransform result = RigidTransform::identity();
47     const std::size_t n = std::min(source_indices.size(), target_indices.size
↳ ());
48     if (n < 3) return result;
49
50     // Центровете се смятат в double. При облак, отместен далече от началото,
51     // сумирането във float губи точно тези значещи цифри, които носят формат
↳ a.
52     double scx = 0.0;
53     double scy = 0.0;
54     double scz = 0.0;
55     double tcx = 0.0;
56     double tcy = 0.0;
57     double tcz = 0.0;
58     for (std::size_t i = 0; i < n; ++i) {
59         const Point3 s = source.point(source_indices[i]);
60         const Point3 t = target.point(target_indices[i]);
61         scx += s.x;
62         scy += s.y;
63         scz += s.z;
64         tcx += t.x;
65         tcy += t.y;
66         tcz += t.z;
67     }
68     const double inv_n = 1.0 / static_cast<double>(n);
69     scx *= inv_n;
70     scy *= inv_n;
71     scz *= inv_n;
72     tcx *= inv_n;
73     tcy *= inv_n;
74     tcz *= inv_n;
75

```

```

76 // Кръстосана ковариация  $H = \sum (s - s_c) * (t - t_c)^T$ .
77 Mat3 h = Mat3::zero();
78 for (std::size_t i = 0; i < n; ++i) {
79     const Point3 s = source.point(source_indices[i]);
80     const Point3 t = target.point(target_indices[i]);
81     const double sx = s.x - scx;
82     const double sy = s.y - scy;
83     const double sz = s.z - scz;
84     const double tx = t.x - tcx;
85     const double ty = t.y - tcy;
86     const double tz = t.z - tcz;
87     h(0, 0) += sx * tx;
88     h(0, 1) += sx * ty;
89     h(0, 2) += sx * tz;
90     h(1, 0) += sy * tx;
91     h(1, 1) += sy * ty;
92     h(1, 2) += sy * tz;
93     h(2, 0) += sz * tx;
94     h(2, 1) += sz * ty;
95     h(2, 2) += sz * tz;
96 }
97
98 Svd3 svd = svd3(h);
99 Mat3 rotation = svd.v * svd.u.transposed();
100
101 // Отражение вместо ротация. Получава се при шум върху почти симетричен
102 // облак и се поправя чрез смяна на знака на стълба, съответстващ на
103 // най-малкото сингулярно число.
104 if (rotation.determinant() < 0.0) {
105     int smallest = 0;
106     for (int j = 1; j < 3; ++j) {
107         if (svd.s[static_cast<std::size_t>(j)] < svd.s[static_cast<std::
↪ size_t>(smallest)]) {
108             smallest = j;
109         }
110     }
111     for (int i = 0; i < 3; ++i) svd.v(i, smallest) = -svd.v(i, smallest);
112     rotation = svd.v * svd.u.transposed();
113 }

```

```

114
115     const Point3 rotated_center = rotation * Point3{static_cast<float>(scx),
↪ static_cast<float>(scy),
116                                                         static_cast<float>(scz)};
117     result.rotation = rotation;
118     result.translation = {static_cast<float>(tcx) - rotated_center.x,
119                         static_cast<float>(tcy) - rotated_center.y,
120                         static_cast<float>(tcz) - rotated_center.z};
121     return result;
122 }

```

Листинг 3: Оценяване на кораво преобразуване по Кабш, `src/icp.cpp`

4.5. Итерация на съвместяването

Функцията `icp_point_to_point` прави на всяка итерация четири неща: намира най-близък съсед в целевия облак за всяка точка от преместения изходен облак, отхвърля съответствията над `max_correspondence_distance`, оценява преобразуване по приетите двойки и го прилага. Прагът за отхвърляне не е украса: без него всяка точка получава съответствие, включително точките без покритие в целевия облак, и те издърпват решението към грешен отговор.

Оценената на всяка стъпка добавка се смята върху *вече преместения* облак, затова се композира *отляво* към натрупаното преобразуване. Обратният ред дава решение, което изглежда правдоподобно, но не съвпада с еталонното преобразуване, и е точно видът грешка, който минава незабелязано без проверка срещу известен отговор.

Спирането е по три условия наведнъж: изчерпани итерации, пренебрежима добавка на последната стъпка, или пренебрежима промяна в отклонението. Отчетеното отклонение е това на *крайното* положение: след цикъла се прави още едно намиране на съответствия, защото отклонението от последната итерация е отпреди прилагането на добавката и не отчита печалбата от нея.

4.6. Прореждане с вокселна решетка

Групирането по кубчета става чрез *сортиране*, а не чрез хеш-таблица. Всяка точка получава 64-битов ключ, в който трите индекса на кубчето заемат по 21 бита,

масивът се сортира по ключ и групите се отчитат при едно обхождане. Изборът е заради повторемостта: хеш-таблицата дава ред на изхода, който зависи от реализацията, а тук две изпълнения върху един и същ вход трябва да дадат еднакъв изход, иначе сравненията между измервания губят смисъл.

Границата от 21 бита на ос, тоест 2 097 152 кубчета, е *проверена*, а не *приета*: при твърде дребно ребро функцията отказва с изключение вместо да върне мълчаливо сгрешен резултат от препълване на ключа.

Изходната точка на кубче е центърът на тежестта на попадналите в него, а не средата на кубчето. Средата внася грешка до половин ребро дори при точки, легнали точно върху равнина, а центърът на тежестта запазва повърхнината.

4.7. Вход и изход

Четците на PLY и на PCD споделят една и съща постановка: заглавието се разбира дотам, докъдето трябва, и от него се извлича *кой стълб* от таблицата в тялото е x , кой y и кой z . Стълбовете се търсят по име, не по позиция, защото и двата формата допускат допълнителни полета и произволен ред. Двоичните варианти се отхвърлят изрично с изключение, а не се четат наполовина. Несъответствието между обявления и действителния брой точки също е грешка.

Четецът на PGM и PPM има един тесен момент. Заглавието допуска коментари навсякъде, включително между размерите, а след максималната стойност стои точно *един* разделител, след който започват суровите байтове. Четенето на лексема затова връща прекратяващия знак в потока: първата реализация го изяждаше и после отхвърляше напълно редовен файл заради „липсващ разделител“.

4.8. Растерни стъпки

Гаусовото ядро се нормира по *действителната* сума на отрязаното ядро, а не по аналитичната константа. При радиус $[3\sigma]$ отрязаната опашка е малка, но не е нула, и нормирането по константата прави изгладеното изображение системно по-тъмно точно с нея. Изглаждането е разделяемо, първо по редове, после по стълбове; цената е пропорционална на $2r$ вместо на $(2r + 1)^2$.

Границата навсякъде се обработва със захващане на координатите към ръба, а не с допълване с нула. Допълването с нула вкарва изкуствен контур по периметъра, който

градиентът после честно намира.

Потискането на немаксимумите закръгля посоката на градиента до едно от четири направления направо от знаците и отношението на компонентите. Изчисляването на atan2 за всеки пиксел е излишно: границите между секторите отговарят на $\tan 22,5^\circ$ и $\tan 67,5^\circ$ и се проверяват с две умножения. Сравнението със съседите е строго от едната страна и нестрого от другата. При плато с равни стойности строгото сравнение и от двете страни изтрива целия контур, а нестрогото и от двете го оставя двоен.

Двупраговото решение и свързаните компоненти ползват *явен* стек и явна опашка, а не рекурсия. Една свързана област може да покрие цялото изображение и дълбочината на рекурсията тогава става равна на броя на пикселите ѝ.

4.9. Проверка и измерване

Проверката е с двадесетина реда собствена рамка в `tests/test_framework.hpp`: регистрация чрез глобален обект, конструиран преди `main`, и четири макроса за твърдения. Групите проверки съответстват на файловете в `tests/` и всяка е отделен запис в `CTest`, като програмата приема името на групата като довод.

Измерването е в `include/pointforge/timing.hpp`: `std::chrono::steady_clock`, загряващи изпълнения, повторения и отчет на минимум, медиана и разсейване. Функцията `keep` пречи на компилатора да изхвърли измервано изчисление, чийто резултат не се ползва; без нея цял измерван цикъл може да изчезне и измерването да покаже нула.

4.10. Едно наблюдение за изграждането

Под MinGW изпълнимите файлове се свързват статично със стандартната библиотека. Причината е наблюдавана, не предпазна: на машина с повече от една инсталация на MinGW програмата зарежда първата `libstdc++-6.dll` от `PATH`, а тя може да е по-стара от компилатора, с който е компилиран проектът. Резултатът е падане още при първото отваряне на файл, което няма нищо общо с кода на проекта и отнема време, за да бъде отнесено на правилното място.

V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА

Измерването отговаря на два отделни въпроса и затова е разделено на две части. Първата е за коректност: дава ли реализацията същия резултат като еталон, за който е известно, че е верен. Втората е за цена: колко време и колко памет отнема. Двете не се смесват, защото по-бърза стъпка с различен резултат не е по-бърза, а друга стъпка.

5.1. Еталон за коректност

За заявките към индекса еталонът е *изчерпателното търсене*: обхождане на всички точки и подреждане по разстояние. То е точно по определение, не изисква външна библиотека и е реализирано в проекта като `brute_force_knn` и `brute_force_radius`. Сравнението е по множеството от върнати разстояния, а не по върнатите номера: при две точки на еднакво разстояние от заявката кой от двата съседа ще влезе в отговора не е част от договора и еталонът има същото право на избор като индекса.

За съвместяването еталонът е *известното преобразуване*. Генераторът произвежда целевия облак, прилага върху него зададено кораво преобразуване и подава на процедурата резултата. Оценява се разликата между възстановеното и зададеното, изразена като ъгъл на относителната ротация и като дължина на разликата на трансформациите.

За останалите стъпки еталонът е свойство, което резултатът трябва да има независимо от реализацията: гаусовото ядро сумира до единица, изглаждането на постоянно изображение не го променя, градиентът на вертикален ръб има нулева съставка по y , потискането на немаксимумите оставя най-много два пиксела напречно на ръб, който преди него е бил широк, прореждането не изнася точка извън ограждащия паралелепипед на входа. Тези проверки са 60 на брой и са част от набора, който CTest пуска.

Сравнение с реализациите на същите алгоритми в утвърдените библиотеки с отворен код беше разгледано и *изоставено*. Причината е ограничението за изграждане, изложено в Раздел III: проектът се компилира с компилатор за C++20 и CMake и с нищо друго, а такова сравнение би добавило задължителна външна зависимост заради една таблица. Загубата е ограничена, защото за въпроса, който работата изследва, тоест цената на заявката за съседи, вътрешното сравнение между двата пътя върху едни и същи данни е по-пряко от сравнение с чужда реализация, която се различава по повече от един избор.

5.2. Данни

Не се използва външен набор от данни. Всичко се произвежда от генератора в `src/synthetic.cpp` по зададено зърно, така че две изпълнения дават един и същ вход. Използват се два вида вход.

Равномерен облак за измерванията на индекса: точки, разхвърляни равномерно в куб със страна 10 единици, при 10 000, 50 000, 200 000 и 1 000 000 точки. Равномерността е нарочна: тя е случаят, в който k-d дървото е най-близо до балансирано, и затова е честният случай за измерване на самата заявка.

Структурирана сцена за съвместяването: равнина $z = 0$ със страна 8 единици, сфера с радиус 1 и куб със страна 1,6, общо 70 000 точки. Сцената нарочно съдържа три различни по форма части. Само равнина не стига: върху равнина съвместяването е недоопределено по две транскации и по една ротация, процедурата „сходи“ към произволна точка от цяло семейство решения и това изглежда като успех, докато не се сравни с еталона.

Исходният облак за съвместяването не е копие на целевия. Той е случайна извадка от 75 % от точките, зашумена независимо и преместена с обратното на зададеното преобразуване. Така двата облака нямат общи точки, което е положението, в което процедурата работи на практика.

За растерния конвейер входът е изкуствено полутоново изображение: слаб наклон на фона, три правоъгълника, два кръга и гаусов шум. Наклонът на фона е нарочен, постоянният фон прави прага тривиален и сегментацията изглежда по-добра, отколкото е.

5.3. Конфигурация на машината

Всички числа в Раздел VI са получени на една машина, чиято конфигурация е дадена в Табл. 2.

Таблица 2. Конфигурация на машината, на която са проведени измерванията

Компонент	Стойност
Процесор	AMD Ryzen 5 3600, микроархитектура Zen 2
Ядра	6 физически, 12 логически
Тактова честота	3950 MHz максимална
Кеш	L2 общо 3072 KB, L3 общо 32768 KB
Оперативна памет	15,9 GB
Набор инструкции	AVX2, без AVX-512
Операционна система	Windows 11 Pro N, компилация 26200
Компилятор	g++ 15.2.0, MinGW-w64, x86_64-posix-seh
Система за изграждане	CMake 4.3.2, Ninja 1.13.2
Основни флагове	-std=c++20 -O3 -Wall -Wextra
Втора конфигурация	същото плюс -march=native, което на тази машина е znver2

Двете конфигурации на компилиране се измерват отделно и се отчитат отделно. Първата е подразбиращата се за проекта и е преносима: изгражданият файл работи на всеки процесор с x86-64. Втората е с пълния набор инструкции на машината и не е преносима, затова съответната настройка на CMake е изключена по подразбиране. Разликата между двете е предмет на измерване, а не предположение: тя показва колко от печалбата идва от ширината на векторния регистър.

5.4. Протокол на изпълнение

Всяко измерване има загряващи изпълнения, чиито времена се изхвърлят, и след тях повторения, чиито времена се пазят. Сравненията между скаларния и пакетния път се правят с 9 повторения и 2 загрявания, останалите с 5 повторения и 1 загряване. Двата пътя се измерват в едно и също изпълнение на програмата, върху едно и също вече построено дърво и върху едни и същи заявки, така че единствената разлика между двете измервания е тялото на цикъла.

Часовникът е `std::chrono::steady_clock`. Изборът пред `system_clock` не е формален: последният може да бъде преместен назад от синхронизация по време на измерването.

Отчита се **минимумът** от повторенията, а медианата и разсейването стоят до него. Изборът е следствие от наблюдение върху самата машина, не от предпочитание. При първите изпълнения разсейването между повторенията достигаше стотици проценти, а

времената за едно и също измерване се различаваха до три пъти между изпълненията на програмата, тоест по ядрата има чужда работа. Външна намеса може само да *удължи* изпълнението, никога да го скъси, затова минимумът е най-близкото до ненарушеното време, а медианата се движи с товара и не е повторяема. Разсейването се отчита именно за да е видимо кое измерване е надеждно и кое не; там, където то е малко, минимумът и медианата съвпадат и изборът няма значение.

Част от измерваните величини изобщо не зависят от товара на машината: брой итерации до сходимост, крайно отклонение, грешка спрямо еталонното преобразуване, брой точки след прореждане, брой възли и заета памет. Те са определени от входа и от алгоритъма, повтарят се точно между изпълненията и се отчитат без уговорка.

Функцията *кеер* в измервателната част пречи на компилатора да изхвърли изчисление, чийто резултат не се ползва. Без нея цял измерван цикъл може да бъде премахнат и измерването да покаже нула.

5.5. Измервани величини и форма на таблиците

Измерват се: време за построяване на индекса и заета от него памет, време на заявка при $k = 1, 8$ и 32 , време на заявка по радиус, влияние на размера на листа, време на изолирано обхождане на лист, време на прореждането, брой итерации и крайно отклонение при съвместяване спрямо нивото на шума, и време на всяка стъпка от растерния конвейер.

Измерването на изолираното обхождане на лист заслужава отделна дума, защото не е очевидно защо съществува. Времето на цялата заявка съдържа и обхождането на дървото, и работата с купчината, а те не зависят от избрания път. Ако разликата между двата пътя е малка спрямо тях, тя не се вижда в общото време, независимо каква е. Затова се прави и второ измерване, при което дървото е построено с размер на листа над броя на точките, тоест цялото дърво е един лист. Тогава всяка заявка е само обхождане на лист. Цената е, че това вече не е заявка, каквато някой би направил на практика, затова резултатът стои като отделна таблица, а не заменя измерването на цялата заявка.

Формата на таблиците е фиксирана предварително и не зависи от това, което измерването е показало. Всички числа в Раздел VI идват от изхода на `pointforge_bench`, който пише и таблица за четене, и файл във формат CSV, точно за да не се преписват стойности на ръка. Записаните изпълнения са в `results/` в хранилището.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя резултатите по методиката от Раздел V. Редът следва реда на въпросите: първо съвпадението с еталона, защото без него измерването на времето няма смисъл, после цената на заявката, после цената на останалите стъпки.

Всички числа идват от два записа: `results/measurements_baseline.txt` за подразбиращата се конфигурация и `results/measurements_native.txt` за конфигурацията с `-march=native`. Съответните файлове във формат CSV са до тях.

6.1. Съвпадение с еталона

Наборът от проверки съдържа 60 случая, разпределени в шест групи, и всички минават. Част от тях са изброени в Табл. 3 заедно с еталона, срещу който сравняват.

Таблица 3. Проверка за коректност: какво се сравнява и срещу какво

Проверявано	Еталон	Резултат
Заявка за k най-близки съседи	изчерпателно търсене, $k = 1, 5, 32, 40$ заявки	съвпадение по множеството от разстояния
Скаларен срещу пакетен път	един срещу друг, 60 заявки	съвпадение до бит, допуск 0
Заявка по радиус	изчерпателно търсене, три радиуса, 30 заявки	съвпадение по множеството от номера
Заявка за единствен съсед	първият резултат от заявката за k	съвпадение до 10^{-6}
Размер на листа	дърво с лист 1 срещу лист 4, 16, 64, 256	еднакъв отговор
Изродени облаци	изчерпателно търсене	еднакъв отговор, построяването приключва
Сингулярно разлагане	USV^T срещу входа	норма на разликата под 10^{-10}
Оценяване по Кабш	зададено преобразуване	ъгъл под 10^{-5} rad, трансляция под 10^{-4}
Кабш върху равнина	детерминанта на ротацията	в интервала (0,99; 1,01), не отражение
Съвместяване без шум	зададено преобразуване	ъгъл под $5 \cdot 10^{-3}$ rad
Кръговрат PLY, PCD, PGM, PPM	входният облак или кадър	съвпадение до 10^{-5} , а за кадрите до бит

Съвпадението на двата пътя е с допуск точно нула, а не приблизително. Това е

по-силно твърдение от „дават близки резултати“ и е нарочно: двата пътя извършват едни и същи операции с плаваща запетая в един и същ ред, само групирани различно, затова всяко разминаване би било грешка, а не закръгляване.

Проверката `icp.kabsch_never_returns_a_reflection` улови действителна грешка в сингулярното разлагане, описана в Раздел IV. Тя е показателна за това защо еталонът трябва да е независим: реализацията изглеждаше вярна и минаваше проверката за възстановяване на матрицата, а грешеше само при вход с ранг под три.

6.2. Построяване на индекса

Табл. 4 дава времето за построяване и заетата памет. Броят възли и паметта не зависят от товара на машината и се повтарят точно.

Таблица 4. Построяване на k-d дървото, размер на листа 32, 5 повторения

Точки	Мин., ms	Мед., ms	Разс., %	Памет, MiB	Възли
10 000	1,379	1,477	7,9	0,18	1 023
50 000	8,440	8,855	11,8	0,88	4 095
200 000	50,659	52,831	10,6	3,53	16 383
1 000 000	432,459	575,300	59,3	17,64	65 535

Паметта расте точно линейно: 17,64 MiB за милион точки са 18,5 байта на точка, при 12 байта координати, 4 байта номер и остатък за възлите. Времето расте по-бързо от линейното: от 50 000 до 1 000 000 точки, тоест двадесеткратно увеличение, времето нараства петдесеткратно. Част от разликата е очакваната от $n \log n$ на `nth_element`, но множителят е по-голям и последният ред е с разсейване 59,3 %, което го прави най-ненадеждния в таблицата. При `-march=native` същият случай дава 352,036 ms, тоест конфигурацията на компилиране влияе и на построяването, макар в него да няма векторизиран цикъл; разликата е в границите на разсейването и не се тълкува.

6.3. Цена на заявката за съсед

Табл. 5 дава времето за 20 000 заявки при трите стойности на k .

Таблица 5. Заявка за k най-близки съседи, 20 000 заявки, 9 повторения, подразбираща се конфигурация

Точки	k	Скал., ms	Пак., ms	Отн.	Разс., %	Заявки/s
10 000	1	8,70	8,48	1,03	16,2	2 357 823
10 000	8	26,81	25,13	1,07	54,7	795 909
10 000	32	85,13	90,71	0,94	171,2	220 495
50 000	1	12,20	11,40	1,07	169,1	1 753 986
50 000	8	111,40	41,47	2,69	215,4	482 243
50 000	32	132,98	107,22	1,24	61,9	186 535
200 000	1	17,76	18,50	0,96	103,4	1 080 818
200 000	8	134,36	50,07	2,68	147,9	399 463
200 000	32	136,71	148,32	0,92	107,9	134 841
1 000 000	1	31,44	31,95	0,98	48,8	626 047
1 000 000	8	65,09	58,88	1,11	38,5	339 655
1 000 000	32	161,22	139,85	1,15	40,6	143 006

Таблицата не позволява извод за отношението между двата пътя и това е самият резултат. Отношението се движи между 0,92 и 2,69, а разсейването в същите редове стига 215 %. Двата реда с отношение около 2,69, при 50 000 и 200 000 точки и $k = 8$, са придружени от най-голямото разсейване в таблицата: там минимумът на скаларния път не е уловил неразтревожено изпълнение. Тези редове не се тълкуват. Същите случаи в конфигурацията с `-march=native` дават 0,88 и 1,13, което потвърждава, че стойността около 2,69 е следствие от товара на машината, а не от кода.

Изводът, който данните поддържат, е отрицателен и е записан като такъв: *разликата между двата пътя в цялата заявка е под разделителната способност на тази машина*. Част от времето на заявката отива в обхождане на дървото и в работа с купчината, а те са еднакви и за двата пътя. Колко точно е тази част, се вижда от следващото измерване.

Пропускателната способност е величина, за която таблицата все пак стига: около 2,4 милиона заявки в секунда при $k = 1$ върху 10 000 точки и около 143 000 при $k = 32$ върху милион точки. Спадът от 10 000 до 1 000 000 точки при $k = 32$ е под два пъти, при стократно увеличение на облака, което съответства на логаритмичната зависимост, очаквана от устройството на дървото.

6.4. Изолирано обхождане на лист

Табл. 6 премахва обхождането на дървото от измерването: дървото е построено с размер на листа над броя на точките, тоест е един лист, и всяка заявка е само обхождане на лист. Общата работа е изравнена между редовете.

Таблица 6. Изолирано обхождане на лист, 9 повторения. Ляво: подразбираща се конфигурация. Дясно: с `-march=native`

Точки в листа	k	Подразбираща се			-march=native		
		Скал., ms	Пак., ms	Отн.	Скал., ms	Пак., ms	Отн.
1 024	1	93,76	37,18	2,52	54,12	23,77	2,28
1 024	8	109,85	82,44	1,33	91,70	70,46	1,30
8 192	1	55,79	26,16	2,13	53,95	17,72	3,05
8 192	8	62,63	30,99	2,02	57,23	24,68	2,32
65 536	1	52,91	22,23	2,38	49,87	15,30	3,26
65 536	8	42,97	18,38	2,34	55,25	23,24	2,38

Тук резултатът е еднозначен. Пакетният път е между 2,0 и 2,5 пъти по-бърз в подразбиращата се конфигурация и между 2,3 и 3,3 пъти при `-march=native`, ако се изключи редът с $k = 8$ и лист от 1 024 точки. Посоката е една и съща във всичките дванадесет клетки, при две независими конфигурации на компилиране.

Печалбата от по-широкия регистър е видима и подредена: при лист от 8 192 и 65 536 точки отношението нараства от 2,13 и 2,38 на 3,05 и 3,26 при преминаване от 16-байтови на 32-байтови вектори. Тя обаче не е двукратна, каквато би била при чисто аритметично ограничение. Причината е, че вторият цикъл, този с проверките и обновяването на купчината, не се векторизира и не се променя от ширината на регистъра.

Редът с лист от 1 024 точки и $k = 8$ се отличава: отношението е 1,33 и 1,30 срещу над 2 навсякъде другаде. При малък лист и голямо k купчината се пълни и остава пълна, тоест делът на неекторизираната работа е най-голям точно там. Това е същият механизъм, който обяснява и Табл. 5.

6.5. Заявка по радиус и размер на листа

Табл. 7 дава заявката по радиус. Средният брой намерени съседи е детерминиран и се повтаря точно между изпълненията.

Таблица 7. Заявка по радиус, 2 000 заявки, подразбираща се конфигурация

Точки	Радиус	Скал., ms	Пак., ms	Отн.	Средно съседи
10 000	0,20	0,52	0,44	1,19	0,3
10 000	0,50	1,01	0,91	1,11	4,9
50 000	0,20	0,85	0,96	0,88	1,6
50 000	0,50	2,37	2,88	0,82	24,5
200 000	0,20	2,56	1,72	1,49	6,5
200 000	0,50	8,40	6,02	1,40	98,5
1 000 000	0,20	6,25	6,05	1,03	32,6
1 000 000	0,50	32,81	43,13	0,76	494,0

Отношението отново се движи около единица и не подкрепя извод. За заявката по радиус това е и очакваното: при нея скъпата част не е сметката на разстоянието, а записването на намерените съседи в изходния вектор, а то е еднакво и за двата пътя. При радиус 0,50 и милион точки на заявка се връщат средно 494 съседи, тоест почти цялата работа е запис.

Табл. 8 дава влиянието на размера на листа при 200 000 точки и $k = 8$.

Таблица 8. Размер на листа, 200 000 точки, $k = 8$, 5 000 заявки

Лист	Подразбираща се			-march=native		
	Постр., ms	Скал., ms	Пак., ms	Постр., ms	Скал., ms	Пак., ms
4	66,03	13,18	13,62	71,52	16,71	14,69
8	53,90	9,66	10,27	60,67	14,05	10,71
16	49,84	8,86	8,71	48,26	8,80	8,63
32	54,84	16,79	17,85	46,51	9,59	12,35
64	74,99	17,22	11,21	72,41	12,05	10,67
128	56,69	15,43	15,48	42,37	14,24	12,20

Двете конфигурации се съгласяват за най-добрия размер на листа: 16 точки, с 8,71 ms и 8,63 ms за пакетния път, съответно 8,86 ms и 8,80 ms за скаларния. Съгласието на две независими изпълнения върху една и съща стойност е по-силен довод от всяко отделно число в таблицата. При лист 4 работата е в обхождането на дървото, при лист 128 в излишно проверени точки, а между двете има минимум.

Времето за построяване спада с растежа на листа, което е очаквано: по-големият лист означава по-плитко дърво и по-малко нива на разделяне.

6.6. Прореждане с вокселна решетка

Табл. 9 дава прореждането на милион точки. Броят изходни точки е детерминиран и съвпада точно между двете конфигурации и между изпълненията.

Таблица 9. Прореждане с вокселна решетка, 1 000 000 точки в куб със страна 10

Ребро	Мин., ms	Мин., ms (native)	Изходни точки	Остават, %
0,020	150,28	293,71	995 925	99,59
0,050	146,82	208,18	940 245	94,02
0,100	131,54	197,65	631 816	63,18
0,250	107,38	155,69	64 000	6,40

Времето е доминирано от сортирането на милион ключа и слабо зависи от реброто: между най-финото и най-едрото прореждане разликата е 1,4 пъти, докато броят изходни точки пада 15 пъти. Това съответства на устройството: сортирането се прави винаги върху целия вход, а от реброто зависи само колко групи ще излязат от него.

Редът с ребро 0,250 е достоен за отбелязване: изходът е точно 64 000 точки, тоест 40^3 , и това е броят на кубчетата, а не на точките. При това ребро всяко кубче в куба със страна 10 съдържа поне една от милиона точки, тоест решетката е наситена.

Числата в конфигурацията с `-march=native` са системно по-големи, което не се тълкува като ефект от компилирането: прореждането няма векторизиран цикъл, а изпълнението с `native` е било второто по ред и е попаднало на по-натоварена машина.

6.7. Съвместяване

Табл. 10 дава поведението на съвместяването спрямо нивото на шума. Всички колони без последната са детерминирани и се повтарят точно между двете конфигурации.

Таблица 10. Съвместяване, точка към точка. Праг за отхвърляне 0,5, най-много 60 итерации. Зададеното преобразуване е ротация от $7,45^\circ$ и трансляция с дължина 0,229

Шум σ	Итерации	Сходимост	Отклонение	Грешка по ъгъл, $^\circ$	Мин., ms
0,000	38	да	0,015230	0,0055	309,01
0,002	39	да	0,015365	0,0016	326,28
0,005	41	да	0,017720	0,0035	380,62
0,010	49	да	0,022771	0,0040	468,24
0,020	60	не	0,030791	0,0430	641,42
0,040	60	не	0,038900	0,8019	829,56

Броят итерации до сходимост расте с шума: 38, 39, 41, 49, и при $\sigma \geq 0,020$ границата от 60 итерации се изчерпва, преди условието за спиране да е изпълнено. Грешката по ъгъл остава под $0,006^\circ$ до $\sigma = 0,010$, тоест до шум, равен на една пета от реброто на прореждащата решетка. При $\sigma = 0,040$ тя скача на $0,80^\circ$, което е двеста пъти повече, макар отклонението да е нараснало само 2,6 пъти. Двете величини измерват различни неща и точно това разминаване го показва: отклонението мери колко близо са точките, а грешката по ъгъл мери дали са в *правилното* положение, и второто се разпада по-рязко.

Отклонението при $\sigma = 0$ не е нула, а 0,0152. Това не е грешка на процедурата, а следствие от постановката: двата облака съдържат *различни* точки, извадка от 75 % срещу пълния набор, прекарани през прореждане с ребро 0,05. Дори при съвършено съвместяване най-близкият съсед на точка от единия облак е на разстояние от порядъка на разстоянието между съседни точки в другия. Стойността 0,0152 е около една трета от реброто на решетката, което е точно очакваният порядък.

Демонстрацията, чийто изход е независим от този експеримент, дава същото поведение при $\sigma = 0,004$: 40 итерации, отклонение 0,016836, грешка по ъгъл $0,002035^\circ$ и грешка по трансляция 0,000158 при трансляция с дължина 0,229, тоест относителна грешка под 0,07 %.

6.8. Растерен конвейер

Табл. 11 дава времената на стъпките върху кадър 1024 на 768, тоест 786 432 пиксела.

Таблица 11. Растерен конвейер, кадър 1024 на 768, 5 повторения, подразбираща се конфигурация

Стъпка	Мин., ms	Мед., ms	ns на пиксел
Гаусово изглаждане, $\sigma = 1,6$	47,305	52,532	60,2
Градиент по Собел	16,375	31,768	20,8
Потискане на немаксимуми	11,225	11,739	14,3
Двупрагово решение	0,707	0,849	0,9
Свързани компоненти	5,925	5,960	7,5

Изглаждането доминира. При $\sigma = 1,6$ радиусът е 5, тоест ядрото е с 11 отчета и разделимото изглаждане прави 22 умножения на пиксел. Градиентът по Собел прави четири преминавания с ядро от 3 отчета, тоест 12 умножения на пиксел, плюс по един корен за големината. Отношението на броя умножения е 1,8, а измереното отношение на

времената е 47,305 към 16,375, тоест 2,9. Разликата между двете не се обяснява от броя операции и не се обяснява тук: причината ѝ не е измерена, а предположение на нейно място би било число без покритие.

Двупраговото решение е с два порядъка по-евтино от останалите стъпки, защото за повечето пиксели се свежда до едно сравнение: проследяването тръгва само от пикселите над горния праг, а те са малка част от кадъра. В демонстрацията, върху кадър 320 на 240, по контур попадат 1 223 пиксела, тоест 1,59 % от кадъра.

6.9. Анализ

Три извода са подкрепени от таблиците и един въпрос остава без отговор.

Първо, векторизацията на обхождането на лист работи и печалбата ѝ е около двукратна. Табл. 6 я показва в дванадесет клетки при две независими конфигурации на компилиране, с една и съща посока навсякъде. Че цикълът наистина се превежда във векторни инструкции, е потвърдено и независимо от времето, чрез отчета на компилатора, цитиран в Раздел IV.

Второ, тази печалба не се вижда в цялата заявка. Табл. 5 не позволява извод: отношението се движи около единица, а разсейването надхвърля разликата, която се търси. Обяснението следва от първите два извода взети заедно. Обхождането на дървото, работата с купчината и записът на резултата не се векторизират, и колкото по-голямо е k , толкова по-голям е делът им. Редът с най-малък лист и най-голямо k в Табл. 6, където отношението пада от над 2 на 1,3, е същият механизъм, видян отвътре.

Това е отклонение от очакваното в Раздел II, където разположението по компонента беше избрано именно заради заявката. Отклонението се отчита като отклонение. Изборът на разположение не е погрешен, той дава двукратно ускорение на частта, която покрива, но частта е по-малка от предполагащото и затова печалбата в целия конвейер е по-малка от очакваната.

Трето, съвместяването възстановява зададеното преобразуване с грешка под $0,006^\circ$ при шум до една пета от реброто на прореждащата решетка, и се разпада предвидимо над това ниво (Табл. 10).

Въпросът, който остава без отговор, е количествен: каква точно е печалбата от векторизацията в цялата заявка. Тя не е нула, защото частичната печалба е измерена, но е под разделителната способност на машината, на която са проведени измерванията.

Отговорът изисква машина без чужд товар по ядрата, а не друга реализация. Това е ограничение на експеримента и е записано като такова, вместо да бъде заместено с правдоподобно число.

ЗАКЛЮЧЕНИЕ

Проектът разглежда конвейер за обработка на облаци от точки и растерни изображения, в който търсенето на съседи е обособено като отделен компонент върху пространствен индекс и е ускорено чрез подредба на данните по компонента и пакетно обхождане на листата. Обособяването има две последици. Първата е инженерна: всяка стъпка ползва един и същ тесен интерфейс, така че реализацията на индекса се сменя без промени в стъпките. Втората е методическа: двата пътя на обхождане са параметър на една и съща заявка, не два различни класа, и затова се сравняват върху едно и също дърво, в едно и също изпълнение, при единствена разлика тялото на цикъла.

Реализацията се компилира с компилатор за C++20 и CMake и с нищо друго. Проверката, 60 случая под CTest, измерването и разлагането на матрица 3×3 , което съвместяването изисква, са написани в проекта именно за да няма задължителна външна зависимост. Това ограничение струва работа, но премахва най-честата причина едно хранилище да не се компилира при първия опит на чужда машина.

Измерените резултати са три, и единият от тях е отрицателен.

Векторизацията на обхождането на лист работи. При изолирано измерване, с дърво от един лист, пакетният път е между 2,0 и 2,5 пъти по-бърз от скаларния в подразбиращата се конфигурация и между 2,3 и 3,3 пъти при пълния набор инструкции на машината, с еднаква посока във всички дванадесет измерени клетки (Табл. 6). Че цикълът се превежда във векторни инструкции, е потвърдено и отделно, чрез отчета на компилатора.

Тази печалба обаче не се вижда в цялата заявка. Табл. 5 не позволява извод: отношението между двата пътя се движи около единица, а разсейването на измерването надхвърля разликата, която се търси. Причината е в дела: обхождането на дървото, работата с купчината от кандидати и записът на резултата не се векторизират, и колкото по-голямо е k , толкова по-голям е техният дял. Това е отклонение от очакването, с което разположението по компонента беше избрано в Раздел II. Изборът не е погрешен, но обхватът на ползата от него е по-тесен от предположения.

Съвместяването възстановява зададеното кораво преобразуване с грешка по ъгъл под $0,006^\circ$ при шум до една пета от реброто на прореждащата решетка, и се разпада предвидимо над това ниво: 38 итерации при нулев шум, 49 при $\sigma = 0,010$, а при $\sigma \geq 0,020$ границата от 60 итерации се изчерпва преди сходимост (Табл. 10).

Ограниченията на решението са три и всичките са известни.

Печалбата от разположението по компонента зависи от вида на натоварването. Изборът е изгоден за конвейер, в който преобладават заявките по околност, и неизгоден за натоварване, в което се четат единични точки в произволен ред, защото достъпът до отделна точка изисква четене от три различни масива.

Измерванията са проведени на машина с чужд товар по ядрата. Затова се отчита минимумът от повторенията, а разсейването стои до всяко число. Величините, които не зависят от товара, тоест брой итерации, отклонение, грешка спрямо еталона, брой точки и заета памет, са надеждни; времената при разсейване над сто процента не са и това е отбелязано на всяко място, където се среща.

Разпознаването по обучени модели остава извън обхвата, по причината, изложена в увода. То е разгледано в литературния преглед и мястото му в конвейера е посочено.

Посоките за продължение са четири. Първата е повтаряне на измерването на цялата заявка на машина без страничен товар, което е единственото, което липсва, за да получи вторият резултат количествен отговор вместо отрицателен. Втората е паралелно изпълнение върху няколко ядра: договорът на стъпката вече е подходящ, понеже стъпките не променят входа си на място. Третата е приблизително търсене на съсед с контролирана загуба на точност [10], разгледано и оставено извън обхвата именно защото внася втори източник на грешка. Четвъртата е добавяне на разпознавател над изхода на конвейера, което би затворило веригата от сензор до решение.

ЛІТЕРАТУРА

- [1] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Sixth International Conference on Computer Vision (ICCV)*, pp. 839–846, 1998.
- [2] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.
- [3] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (FPFH) for 3D registration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3212–3217, 2009.
- [4] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [5] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural features for image classification,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973.
- [6] H. Hirschmüller, “Stereo processing by semiglobal matching and mutual information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008.
- [7] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.
- [8] S. Rusinkiewicz and M. Levoy, “Efficient variants of the ICP algorithm,” in *Proceedings of the Third International Conference on 3-D Digital Imaging and Modeling*, pp. 145–152, 2001.
- [9] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [10] M. Muja and D. G. Lowe, “Scalable nearest neighbor algorithms for high dimensional data,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 11, pp. 2227–2240, 2014.

- [11] R. B. Rusu and S. Cousins, “3D is here: Point Cloud Library (PCL),” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–4, 2011.
- [12] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

ПРИЛОЖЕНИЯ

Приложение А. Структура на изходния текст

pointforge/	
CMakeLists.txt	библиотека, демонстрация, измервания, проверки
include/pointforge/	публични типове
point_cloud.hpp	облак от точки, по компонента
transform.hpp	матрица 3x3, разлагане, кораво преобразуване
kdtree.hpp	интерфейс на индекса, двата пътя на обхождане
icp.hpp	съвместяване, точка към точка
voxel_grid.hpp	прореждане с вокселна решетка
cloud_io.hpp	PLY и PCD
image.hpp	Image8 и ImageF
image_io.hpp	PGM и PPM
image_ops.hpp	изглаждане, градиент, немаксимуми, прагове
labeling.hpp	свързани компоненти
synthetic.hpp	генератор на изкуствени данни
timing.hpp	измерване на време
text.hpp	подравняване на изхода
src/	реализация, по един файл на модул
apps/demo.cpp	демонстрация: една команда, видим изход
benchmarks/bench.cpp	измервания, изход и в CSV
tests/	проверки: рамка и по един файл на група
test_framework.hpp	регистрация и твърдения
test_main.cpp	изпълнение и избор на група
test_geometry.cpp	разлагане, ротации, преобразувания
test_cloud_io.cpp	PLY и PCD, и повредени файлове
test_kdtree.cpp	индекс срещу изчерпателно търсене
test_voxel_grid.cpp	прореждане
test_icp.cpp	съвместяване срещу известно преобразуване
test_image.cpp	растерни стъпки и сегментация
results/	записаните изпълнения на измерванията
docs/	този документ

Приложение Б. Формати на структурите от данни

Облак от точки. Три отделни непрекъснати масива от float, по един на координата, всеки с дължина n . Заеманата памет е $12n$ байта плюс запасът на трите вектора. Достъпът до отделна точка събира тройката от трите масива.

Възел на дървото. Полетата са дадени в Табл. 12. Общият размер е 20 байта. Стойност -1 в полето за ос означава лист; тогава двата номера ограждат диапазона от

точки в разбърканите масиви. Левият наследник е винаги следващият възел в масива, затова номер за него не се пази.

Таблица 12. Полета на възела на k-d дървото

Поле	Тип	Байта	Съдържание
split	float	4	стойност на разделящата равнина
axis	std::int32_t	4	0, 1 или 2; -1 за лист
begin	std::uint32_t	4	начало на диапазона в листа
end	std::uint32_t	4	край на диапазона в листа
right_child	std::uint32_t	4	номер на десния наследник

Индексът пази освен възлите и три масива с разбърканите координати, плюс масив с първоначалните номера на точките. За милион точки при размер на листа 32 това дава 65 535 възела и 17,64 MiB общо, тоест 18,5 байта на точка (Табл. 4).

Растрерни типове. Image8 пази 8 бита на канал, с 1 или 3 канала, преплетени по пиксел. ImageF пази по едно число с плаваща запетая на пиксел. Целият път от изглаждането до потискането на немаксимумите тече в ImageF, защото градиентите излизат извън диапазона 0..255 и закръгляването между стъпките би трупало грешка.

Ключ на вокселната решетка. Един 64-битов ключ, в който трите индекса на кубчето заемат по 21 бита. Границата от 2^{21} кубчета на ос е проверена при изпълнение и надхвърлянето ѝ е грешка, не мълчаливо препълване.

Приложение В. Избрани части от изходния текст

Листингите на скаларния и на пакетния път на обхождане, както и на оценяването на преобразуванието по Кабш, са в Раздел IV, съответно Листинг 1, Листинг 2 и Листинг 3. Тук са дадени интерфейсите на индекса и измервателната част, вмъкнати направо от изходните файлове.

```

1 // kdtree.hpp: пространствен индекс над облак от точки.
2 //
3 // Интерфейсът е умишлено тесен: построяване, заявка за k най-близки съседни,
4 // заявка по радиус и отчет за паметта. Всичко останало в проекта е потребите
  ↪ л
5 // на този интерфейс.
```

```

6 //
7 // Двата пътя за изпълнение на листата, скаларен и пакетен, са на разположени
  ↪ е
8 // през един и същ метод и се избират с параметър. Така една и съща заявка се
9 // изпълнява и по двата начина върху едни и същи данни, което е условието
10 // сравнението между тях да е измерване, а не твърдение.
11 #ifndef POINTFORGE_KDTREE_HPP
12 #define POINTFORGE_KDTREE_HPP
13
14 #include <cstddef>
15 #include <cstdint>
16 #include <vector>
17
18 #include "pointforge/point_cloud.hpp"
19
20 namespace pointforge {
21
22 // Съсед: индекс В ИЗХОДНИЯ облак и квадрат на разстоянието. Квадратът, а не
23 // разстоянието, за да няма корен във вътрешния цикъл.
24 struct Neighbor {
25     std::uint32_t index = 0;
26     float squared_distance = 0.0F;
27 };
28
29 // Кой път се ползва при обхождането на лист.
30 enum class NnPath {
31     Scalar,    // точка по точка, с проверка след всяка
32     Batched,   // на блокове от kBlockSize, разстоянията се смятат без разклон
  ↪ ения
33 };
34
35 struct KdTreeOptions {
36     // Максимален брой точки в лист. Балансира дълбочината на обхождането сре
  ↪ шу
37     // дължината на векторизирания цикъл, затова е настройка, а не константа.
38     std::uint32_t leaf_size = 32;
39 };
40
41 class KdTree {

```

```

42 public:
43     // Дължина на блока в пакетния път. Кратна на ширината на векторния
44     // регистър при float за AVX2 (8) и за SSE (4).
45     static constexpr std::uint32_t kBlockSize = 16;
46
47     KdTree() = default;
48     explicit KdTree(const PointCloud& cloud, KdTreeOptions options = {});
49
50     void build(const PointCloud& cloud, KdTreeOptions options = {});
51
52     std::size_t size() const { return px_.size(); }
53     bool empty() const { return px_.empty(); }
54     std::size_t node_count() const { return nodes_.size(); }
55     std::uint32_t leaf_size() const { return options_.leaf_size; }
56
57     // k най-близки съседи на query. Резултатът е подреден по нарастващо
58     // разстояние. Върнатите индекси сочат в облака, подаден на build.
59     void knn(const Point3& query, std::size_t k, std::vector<Neighbor>& out,
60             NnPath path = NnPath::Batched) const;
61
62     std::vector<Neighbor> knn(const Point3& query, std::size_t k, NnPath path
63     ↪ = NnPath::Batched) const;
64
65     // Всички съседи на разстояние до radius включително. Резултатът НЕ е
66     // подреден: подреждането не е част от договора и струва време, което
67     // повечето потребители не искат да платят.
68     void radius_search(const Point3& query, float radius, std::vector<
69     ↪ Neighbor>& out,
70                     NnPath path = NnPath::Batched) const;
71
72     std::vector<Neighbor> radius_search(const Point3& query, float radius,
73                                         NnPath path = NnPath::Batched) const;
74
75     // Единственият най-близък съсед. Отделен път, защото ICP го вика веднъж
76     ↪ на
77     // точка на итерация и купчината за k = 1 е чиста излишна работа.
78     // Връща false само за празно дърво.
79     bool nearest(const Point3& query, Neighbor& out, NnPath path = NnPath::
80     ↪ Batched) const;

```

```

77
78 // Памет, заета от индекса: разбърканите координати, картата на индексите
↪ и
79 // възлите. Отчита се capacity, не size, защото това е заетото.
80 std::size_t memory_bytes() const;
81
82 private:
83     struct Node {
84         float split = 0.0F;
85         std::int32_t axis = -1; // -1 означава лист
86         std::uint32_t begin = 0;
87         std::uint32_t end = 0;
88         std::uint32_t right_child = 0; // левият е винаги следващият в масив
↪ a
89     };
90
91     std::uint32_t build_recursive(std::vector<std::uint32_t>& order, std:::
↪ uint32_t begin,
92                                     std::uint32_t end, const PointCloud& cloud)
↪ ;
93
94     // Координати, пренаредени в реда на обхождане на дървото, така че точките
↪ e
95     // на един лист да са съседни в паметта. Това е предпоставката пакетният
96     // път изобщо да има смисъл.
97     std::vector<float> px_;
98     std::vector<float> py_;
99     std::vector<float> pz_;
100     std::vector<std::uint32_t> original_index_;
101     std::vector<Node> nodes_;
102     KdTreeOptions options_{};
103 };
104
105 // Изчерпателно търсене. Точно по определение, служи за еталон при проверката
106 // на индекса и е бавно нарочно.
107 std::vector<Neighbor> brute_force_knn(const PointCloud& cloud, const Point3&
↪ query, std::size_t k);
108 std::vector<Neighbor> brute_force_radius(const PointCloud& cloud, const
↪ Point3& query, float radius);

```

```

109
110 } // namespace pointforge
111
112 #endif // POINTFORGE_KDTREE_HPP

```

Листинг 4: Интерфейс на пространствения индекс, `include/pointforge/kdtree.hpp`

```

1 // timing.hpp: измерване на време без външна библиотека.
2 //
3 // Часовникът е std::chrono::steady_clock, защото system_clock може да бъде
4 // преместен назад от синхронизация по време на измерването.
5 //
6 // Отчита се медианата, а не средното. Планировчикът на операционната система
7 // произвежда отделни много бавни изпълнения, а средното ги пренася върху цел
8 ↪ ия
9 // резултат. Затова се пазят и минимумът, и максимумът: разликата между тях
10 // казва дали измерването изобщо е стабилно.
11 #ifndef POINTFORGE_TIMING_HPP
12 #define POINTFORGE_TIMING_HPP
13
14 #include <algorithm>
15 #include <chrono>
16 #include <cstdint>
17 #include <vector>
18
19 namespace pointforge {
20 struct TimingStats {
21     double median_ms = 0.0;
22     double mean_ms = 0.0;
23     double min_ms = 0.0;
24     double max_ms = 0.0;
25     int repetitions = 0;
26
27     // Разсейване, отнесено към медианата. Над няколко процента резултатът не
28     // е готов за таблица.
29     double spread_ratio() const {
30         return median_ms > 0.0 ? (max_ms - min_ms) / median_ms : 0.0;

```

```

31     }
32 };
33
34 // Пречи на компилатора да изхвърли изчисление, чийто резултат не се ползва.
35 // Без това цял измерван цикъл може да изчезне и измерването да покаже нула.
36 template <class T>
37 inline void keep(const T& value) {
38     // Празен asm блок с вход, но без изход, кара компилатора да смята
39     // стойността за нужна, без да добавя инструкции.
40 #if defined(__GNUC__) || defined(__clang__)
41     asm volatile("" : : "r,m"(value) : "memory");
42 #else
43     volatile const T sink = value;
44     (void)sink;
45 #endif
46 }
47
48 template <class Body>
49 TimingStats measure(int repetitions, int warmup, Body&& body) {
50     for (int i = 0; i < warmup; ++i) body();
51
52     std::vector<double> samples;
53     samples.reserve(static_cast<std::size_t>(std::max(repetitions, 0)));
54     for (int i = 0; i < repetitions; ++i) {
55         const auto start = std::chrono::steady_clock::now();
56         body();
57         const auto stop = std::chrono::steady_clock::now();
58         samples.push_back(std::chrono::duration<double, std::milli>(stop -
↵ start).count());
59     }
60
61     TimingStats stats;
62     if (samples.empty()) return stats;
63
64     std::sort(samples.begin(), samples.end());
65     stats.repetitions = repetitions;
66     stats.min_ms = samples.front();
67     stats.max_ms = samples.back();
68     const std::size_t mid = samples.size() / 2;

```



```

69     stats.median_ms = (samples.size() % 2 == 1)
70                       ? samples[mid]
71                       : 0.5 * (samples[mid - 1] + samples[mid]);
72     double sum = 0.0;
73     for (const double s : samples) sum += s;
74     stats.mean_ms = sum / static_cast<double>(samples.size());
75     return stats;
76 }
77
78 } // namespace pointforge
79
80 #endif // POINTFORGE_TIMING_HPP

```

Листинг 5: Измерване на време, include/pointforge/timing.hpp

Приложение Г. Как се възпроизвеждат измерванията

Инструментите и версиите им са в Табл. 2. Командите по-долу са тези, с които са получени резултатите от Раздел VI, и са изпълнявани от корена на хранилището.

Изграждане и проверка:

```

cmake -S . -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
cmake --build build
ctest --test-dir build --output-on-failure

```

Демонстрация. Записва три файла PLY и седем файла PGM и PPM в build/demo-output/:

```
cmake --build build --target demo
```

Измервания в подразбиращата се конфигурация:

```
./build/pointforge_bench --csv results/measurements_baseline.csv
```

Измервания с пълния набор инструкции на машината. Изгражданият файл не е преносим, затова настройката е изключена по подразбиране:

```

cmake -S . -B build-native -G Ninja -DCMAKE_BUILD_TYPE=Release \
      -DPOINTFORGE_NATIVE_ARCH=ON
cmake --build build-native
./build-native/pointforge_bench --csv results/measurements_native.csv

```

Проверка кои цикли компилаторът векторизира:

```
g++ -std=c++20 -O3 -Iinclude -c src/kdtree.cpp \  
-o /dev/null -fopt-info-vec  
g++ -std=c++20 -O3 -march=native -Iinclude -c src/kdtree.cpp \  
-o /dev/null -fopt-info-vec
```

Изграждане на този документ:

```
cd docs && latexmk -pdf Main.tex
```